Cortex v1.1.0: facts and original evidence

Local session tests and a bounded memory package

Brendan Playford

21 October 2026
Paper revision 2.2

Software: `v1.1.0`
Code commit: `73c41b2f40516345bbec2bde2429aaf69d68699a`

Abstract

Cortex is a Go package that turns attributed observations into persistent memories and retrieves them for a host application. Release v1.1.0 adds a general fact type, explicit retrieval of original observations, and deletion of observations that consolidation did not retain as memories. It also reports unreferenced inputs and allows one bounded repair of a model response that violates the output contract. This paper describes the changes and evaluates them with synthetic cases, selected local Codex and Claude user messages, and a small public-data smoke test. On 19 questions about selected local sessions, expected-answer checks passed for 14 memory-only answers and all 19 source-based answers. Original evidence remains separate from validated memory; the application decides when to retrieve it and how much to give its reader. These diagnostics informed development. They do not establish general accuracy or superiority over other memory systems.

1 Why this revision exists

The March architecture paper proposed scheduled consolidation, retrieval, forgetting and future learned policies [1]. Cortex implements a smaller package whose host owns scheduling, source consent and authorization. Paper revision 2.1 documented v1.0.1 and proposed an evaluation plan [2]. This revision reports the next implementation step.

The earlier diagnostic exposed omissions during consolidation. Searching stored summaries more accurately cannot recover information that no summary retained. Two changes address that problem: an explicit place for factual statements, and a separate route back to source observations. Neither change guarantees that every useful statement becomes a memory.

The package continues to expose explicit operations. It does not add a scheduler, resident worker, graph database, or model-training service. The software release and paper revision are versioned separately.

2 Implementation

2.1 Facts, summaries and output repair

The six memory kinds are fact, behavior, preference, relationship, project and routine. The new fact kind accommodates explicit statements that do not describe a preference or repeated

behavior. Classification does not make a statement true: it must still cite observations in the current consolidation window.

The consolidation prompt asks the model to check each observation for separate useful facts. Several operations can cite one observation. An update to one fact must not discard other facts in the same message. Summaries remain bounded, now at 512 Unicode characters. The previous 240-character cap rejected some qualified project descriptions in the local tests. The larger cap preserves room for those qualifications while the complete model request remains limited to 1 MiB.

An invalid successful model response receives at most one format-repair call. The original context and rejected response are supplied with an explicit repair contract. The repaired response goes through the same strict parser and transaction validation. Transport failures are not automatically repeated. Failed repair leaves the chunk pending. Token accounting for a completed chunk includes both calls; `format_repairs` reports repair attempts in the request.

Consolidation also returns `unreferenced_observation_ids`. These are processed inputs that supported no operation, not proven omissions. An irrelevant message can correctly remain unreferenced, and a cited multi-fact message can still lose one of its facts. This field supports inspection without pretending to measure semantic coverage.

2.2 Original-observation retrieval

`Recall` retains its memory-only contract. The new `RecallObservations` operation returns original content, source, timestamp and speaker references as a distinct result type. A host can use this evidence when a question cannot be answered from summaries, or compare the two representations under its own reader budget.

`IndexObservations` separately opts raw text into the configured embedding provider. Ordinary memory indexing never sends raw conversations for embedding. Both paths use lexical token overlap and, when configured, exact cosine similarity with reciprocal rank fusion:

$$R_i = \sum_c \frac{1}{60 + \mathrm{rank}_c(i)}.$$

The library does not silently combine raw observations with memories or treat original text as a verified fact. The local comparison in this paper interleaves the two result lists in application code.

Original-observation search is bounded to 20,000 eligible observations per user. Memory search retains its 10,000-active-memory limit. Exceeding a bound returns an error. Embedding records include model identity and content hash. Index publication and retrieval recheck source eligibility after provider work. A missing or failed embedding is visible; callers explicitly choose lexical-only retrieval.

2.3 Lifecycle and deletion

Raw retrieval excludes an observation if it supports any superseded, archived or expired memory. When one observation supports several facts, this conservative rule can suppress a still-useful fact as well. It prevents the source path from bypassing an explicit memory lifecycle decision.

An unassociated observation can still contain outdated information, quotations or an assistant suggestion. The reader must examine its timestamp and attribution. Eligibility is not a guarantee of current truth.

`ForgetObservation` removes original evidence even when consolidation produced no memory. It uses the same per-user lock, import barrier and exact-replay tombstones as memory forgetting.

Exact-content copies, derived memories, connected versions, embeddings and relevant feedback are removed. Deleting an absent observation is an idempotent no-op. Paraphrased future input is outside the exact-replay guarantee.

Confidence continues to represent support from evidence. Elapsed time changes activation rather than confidence. The existing configurable decay, pinning, expiry, maintenance and explicit outcome utility remain unchanged. Reading evidence does not create a reinforcement or training outcome.

3 Evaluation protocol

3.1 Synthetic control

A frozen set contained 24 expected facts in multi-fact messages, explicit corrections, quoted material and instruction-injection canaries. It was defined before implementation and run three times per configuration. The control compared the prior release with the final candidate using Luna 6 medium. Expected-value presence in current summaries was the metric, not generated-answer accuracy.

Both configurations retained all 24 expected values in all three runs and retained none of the introduced canaries. This control therefore gives no evidence of improved extraction coverage from the new fact kind. It checks that the changes preserved this small set of previously working cases.

3.2 Local Codex and Claude sessions

With the owner's permission, the diagnostic used 51 selected user-role messages from five local Codex and Claude sessions. System prompts, tool outputs, interface events and messages containing apparent credentials were excluded. Some user messages contain explicitly attributed quotations. The sample was not random: it used recent eligible messages of bounded length and included project requests as well as durable preferences. Each source conversation had a separate test user so unrelated projects could not supply each other's answers.

Nineteen questions and expected answer patterns were recorded before consolidation. The comparison used memory-only retrieval, raw-observation retrieval, and an alternating combination. Each reader received at most 1,200 tokens of serialized evidence measured with `cl100k_base`. This is a fixed budgeting tokenizer, not a claim about the provider's internal tokenizer. The reader was `gpt-6-luna-medium`, which returned the model identifier `gpt-6-luna`. Pattern checks on short answers are an inspection aid; they do not prove that every qualification or attribution is correct.

The gateway and PostgreSQL 17 ran locally in Docker. The gateway used its existing account pool. The test did not deliberately invalidate live credentials to force an account switch. A local ONNX encoder used `sentence-transformers/all-MiniLM-L6-v2`, producing 384-dimensional vectors.[1] It truncates input at 256 wordpieces, which can miss evidence late in a long observation.

Raw sessions, extracted inputs, model responses, question annotations and database exports remain outside Git. Only methods and aggregate results are published. This protects the owner's data but prevents public reproduction of the private diagnostic.

[1] Pinned encoder revision: `1110a243fdf4706b3f48f1d95db1a4f5529b4d41`.

3.3 Public-data smoke test

The public probe uses the cleaned LongMemEval-S dataset [3, 4]. It selects the first file-order case from each of the six question types and a separate abstention case, seven cases in total. Each case retains its full supplied history. Answer labels and evidence-location annotations are withheld from observation ingestion and used only for evaluation.

Retrieval modes and the reader budget match the local comparison. The judge uses the official answer-checking prompt with Luna 6 medium. The official example evaluator uses other named judge models, so this is a modified protocol. The reader and judge use the same model family, which can introduce shared biases. A seven-case development probe cannot be reported as a full LongMemEval score or support a statistical comparison.

4 Measured results

4.1 Local answer checks

Configuration	Expected-answer checks passed
v1.0.1 memories	7/19
v1.1.0 memories	14/19
v1.1.0 original observations	19/19
v1.1.0 interleaved memories and observations	19/19

Table 1: One reader run per question on the selected local sessions. The checks require expected answer strings; they are not a complete semantic or attribution assessment.

Both configurations completed all five conversations in the reported comparison. The final candidate made three format-repair calls. An earlier baseline attempt failed on the third conversation, and earlier candidate runs also failed on overlong or malformed summaries. These observations motivated the larger summary bound and bounded repair; they also show that hosted-model variation matters.

The five checks missed by the final memory-only path were answered from original observations. They concerned details that a general personal-memory prompt can omit as one-off requests or project context. This supports exposing a separate source path for applications that need those details. It does not show that every temporary request should become a durable memory.

The change from 7 to 14 memory-only checks is descriptive. There was no ablation that isolated the new fact kind from the prompt changes, summary bound or model variation. The 19/19 source result should also not be read as perfect answer quality: the checks do not reject every unnecessary addition or misplaced qualification.

4.2 Public-data results

All seven histories completed consolidation. All three retrieval modes missed the multi-session counting and elapsed-time cases. The original-observation path answered the direct factual and assistant-information cases missed by memory-only retrieval. This suggests a useful role for access to source text, while leaving clear gaps in collecting and combining evidence. The probe does not separate retrieval failure from reader failure, and its small selected sample cannot establish which mode is generally better.

Case	Memories	Observations	Combined
User fact	Fail	Pass	Pass
Abstention	Pass	Pass	Pass
Multi-session counting	Fail	Fail	Fail
Preference	Pass	Pass	Pass
Elapsed time	Fail	Fail	Fail
Knowledge update	Pass	Pass	Pass
Assistant information	Fail	Pass	Pass
Total	3/7	5/7	5/7

Table 2: Seven LongMemEval-S development cases, judged with the official answer-checking prompt and Luna 6 medium. These are not full-benchmark scores.

5 Regression checks and release boundary

The PostgreSQL tests cover the new fact kind, source attribution, explicit indexing, user isolation, capacity bounds, and exclusion of archived, expired and superseded evidence. They also exercise deletion during retrieval and index publication, exact reimport after forgetting, deletion of unretained sources, and removal of derived memories.

Format-repair tests verify a single additional call, combined token usage, unchanged pending inputs after failure, and no automatic transport retry. Existing tests retain the consolidation transaction, correction, feedback and activation contracts. The final race suite, vet, build, source vulnerability scan and secret scan passed. Reimporting all five local session samples inserted zero new observations; each repeated consolidation returned a no-op. Native release startup and the new source-retrieval route passed a smoke check.

Migration 005 adds optional observation embeddings with cascading deletion and a source lookup index. It is additive. Every writer must understand the fact kind before writing new memories, and lifecycle operations still require writers that enforce the earlier tombstones and archive state. The trusted-localhost HTTP service continues to require host-level authentication and authorization.

6 Limits and next work

The experiments are small development diagnostics. They were used to find and correct failures, the private questions share one owner's work context, and the public sample is selected by file order. No result establishes performance on all 500 LongMemEval questions or on other users.

Original evidence can recover details omitted by consolidation, but also increases the reader's exposure to irrelevant, quoted or obsolete text. The next comparison should use a frozen larger benchmark set and report answer correctness, abstention and obsolete-fact use separately. Long observations need query-aware excerpts or separately indexed windows before assuming a single truncated embedding is sufficient. Memory and raw-evidence budget allocation should also be compared directly.

Outcome feedback remains a stored utility signal rather than a learned policy. The package should add more infrastructure only after measurements show which failure it would address.

References

[1] B. Playford. *Applying Sleep-Dependent Memory Consolidation to Agent Architecture*. March 2026. Archived paper version 1. https://github.com/teslashibe/cortex/tree/main/docs/papers/v1-march-2026.

[2] B. Playford. *Evidence-grounded memory in a small Go package*. Paper revision 2.1, October 2026. https://github.com/teslashibe/cortex/tree/main/docs/papers/v2.1-october-2026.

[3] D. Wu et al. *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*. ICLR 2025. https://arxiv.org/abs/2410.10813.

[4] LongMemEval authors. Official benchmark code and cleaned data. Accessed 21 October 2026. https://github.com/xiaowu0162/LongMemEval.

[5] B. Playford. *Cortex v1.1.0*. https://github.com/teslashibe/cortex/releases/tag/v1.1.0.